UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Ingeniería en Tecnologías de la Información e Innovación Digital

---

ACTIVIDAD PARCIAL 2
FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL

Sección 3 · Tema 2.3 Búsqueda Local

Tema: Optimización con Hill Climbing y Simulated Annealing aplicados al problema del viajante (TSP) y a la función Rastrigin

---

Alumna	Dulce María Sánchez Fragozo
Asignatura	Fundamentos de Inteligencia Artificial
Docente	Mtro. Javier García Lara
Grupo / Cuatrimestre	171 · Cuatrimestre 7
Periodo	12 al 26 de octubre de 2026
Sección	3 de 6 · Individual · 50 pts
Referencia	Wirsansky, cap. 2
Repositorio GitHub	[pegar aquí el enlace a la carpeta S3 del repositorio]

1. Tema y planteamiento del problema

Tema. Búsqueda local para optimización: implementación y comparación de *Hill Climbing* (HC) y *Simulated Annealing* (SA) sobre una función objetivo real (Russell & Norvig, 2020; Wirsansky, 2020).

Planteamiento. Un repartidor debe visitar 25 puntos de una zona de 100×100 unidades y regresar al punto de partida recorriendo la menor distancia posible (problema del viajante, TSP). El número de rutas posibles es $(25-1)!/2 \approx 3.1\times10^{22}$; recorrerlas todas es inviable, por lo que se recurre a búsqueda local, que parte de una solución completa y la mejora paso a paso explorando soluciones vecinas. Como segundo problema, más pequeño y continuo, se minimiza la función de Rastrigin en 2 dimensiones, que tiene muchos mínimos locales y sirve para estudiar el efecto del tamaño de paso η.

2. Datos del problema y formulación

Elemento	Problema 1: TSP	Problema 2: Rastrigin 2D
Estado	Permutación de las 25 ciudades (ciclo cerrado)	Vector $x = (x_1, x_2)$, $x_i \in [-5.12, 5.12]$
Función objetivo	$f(\text{ruta}) = \Sigma\, d(c_i, c_{i+1}) + d(c_{25}, c_1)$, minimizar	$f(x) = 20 + \Sigma\,(x_i^2 - 10\cos(2\pi\, x_i))$, minimizar
Vecindad	2-opt: invertir el tramo ruta[i..j]	Perturbación gaussiana $N(0, \eta^2)$ por coordenada
Datos de entrada	25 ciudades con coordenadas uniformes en $[0,100]^2$, semilla 171	Punto inicial uniforme en el dominio
Óptimo conocido	Desconocido (mejor hallado: 420.37)	$f(0,0) = 0$; mínimos locales cerca de 0.995

En el TSP, el costo de un vecino 2-opt se calcula en O(1) a partir de las cuatro aristas que cambian, sin recalcular toda la ruta: $\Delta = d(a,c) + d(b,d) - d(a,b) - d(c,d)$.

3. Procedimiento y algoritmos

Hill Climbing (ascenso más pronunciado)

- Generar una ruta inicial aleatoria.
- Evaluar los 276 vecinos 2-opt (pares i<j; Croes, 1958) y elegir el de mayor mejora.
- Si ninguno mejora, se llegó a un óptimo local y termina; si no, aplicar el movimiento y repetir.
- Variante HC con 10 reinicios aleatorios: se conserva la mejor ruta entre 10 ejecuciones.

Simulated Annealing (Kirkpatrick et al., 1983)

- Ruta inicial aleatoria y temperatura $T = T_0$.
- En cada temperatura se prueban 50 vecinos 2-opt aleatorios. Un vecino que mejora siempre se acepta; uno peor se acepta con probabilidad $\exp(-\Delta/T)$.
- Enfriamiento geométrico $T \leftarrow \alpha \cdot T$ hasta $T < T_min$. Se guarda la mejor ruta vista.
- Parámetros base: $T_0 = 100$, $\alpha = 0.995$, $T_min = 10^{-3}$, 50 iteraciones por temperatura (≈ 114 850 evaluaciones).

Para Rastrigin se usan las mismas ideas con paso gaussiano de desviación η: HC acepta solo mejoras (5 000 iteraciones) y SA usa $T_0 = 10$ y $\alpha = 0.999$.

Análisis de complejidad

- **HC, por iteración:** $O(n^2)$ vecinos con costo O(1) cada uno. En promedio converge en unas 20 iteraciones (≈ 5630 evaluaciones).

- **SA:** el número de evaluaciones es $k \cdot \lceil \ln(T_{min}/T_0)/\ln \alpha \rceil = 50 \times 2\,297 \approx 114\,850$, independiente del tamaño del espacio. Cada movimiento aceptado cuesta O(n) por la inversión del tramo.
- **Memoria:** $O(n^2)$ por la matriz de distancias; ambos algoritmos mantienen una sola solución en curso.

4. Código Python

Archivo `busqueda_local.py` (Python 3, NumPy; Harris et al., 2020, y Matplotlib; Hunter, 2007). Se ejecuta con `python busqueda_local.py` y genera la carpeta `resultados/` con las gráficas y `resultados.json`. Las semillas están fijas, por lo que los resultados son reproducibles.

```
"""
Sección 3 - Búsqueda Local: Hill Climbing y Simulated Annealing
Fundamentos de Inteligencia Artificial - UPA - Grupo 171

Problema 1 (principal): TSP (ruta mínima) con 25 ciudades, vecindad 2-opt.
Problema 2 (complementario): función Rastrigin 2D, con paso de vecindad eta (η).

Uso:  python busqueda_local.py
Salida: carpeta resultados/ con gráficas (PNG) y resultados.json
"""
import json
import math
import os
import random
import time

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

SEMILLA = 171
N_CIUDADES = 25
N_CORRIDAS = 30
OUT = os.path.join(os.path.dirname(os.path.abspath(__file__)), "resultados")
os.makedirs(OUT, exist_ok=True)

AZUL, VERDE, GRIS = "black", "0.35", "0.6"


# ---------------------------------------------------------------- TSP
def generar_ciudades(n, semilla=SEMILLA):
    rng = np.random.default_rng(semilla)
    return rng.uniform(0, 100, size=(n, 2))


def matriz_distancias(c):
    d = c[:, None, :] - c[None, :, :]
    return np.sqrt((d ** 2).sum(-1))


def costo(ruta, D):
    """Función objetivo: longitud total del ciclo hamiltoniano."""
    return sum(D[ruta[i], ruta[(i + 1) % len(ruta)]] for i in range(len(ruta)))


def delta_2opt(ruta, i, j, D):
    """Cambio de costo al invertir el segmento ruta[i..j] (O(1))."""
    n = len(ruta)
    a, b = ruta[i - 1], ruta[i]
    c, d = ruta[j], ruta[(j + 1) % n]
    return D[a, c] + D[b, d] - D[a, b] - D[c, d]


def vecino_aleatorio(ruta, D, rng):
    n = len(ruta)
    i, j = sorted(rng.sample(range(1, n), 2))
    return i, j, delta_2opt(ruta, i, j, D)


def aplicar_2opt(ruta, i, j):
    ruta[i:j + 1] = reversed(ruta[i:j + 1])


def hill_climbing_tsp(D, rng, max_iter=20000, reinicios=1):
    """Hill Climbing de ascenso más pronunciado (steepest-ascent) con 2-opt.
    Con reinicios>1 se convierte en Hill Climbing con reinicio aleatorio."""
    n = len(D)
    mejor_global, hist_global = None, []
    mejor_costo = float("inf")
    evals = 0
    for _ in range(reinicios):
        ruta = list(range(n))
        rng.shuffle(ruta)
        c = costo(ruta, D)
        hist = [c]
```

```
        for _ in range(max_iter):
            mejor_d, mi, mj = 0.0, None, None
            for i in range(1, n - 1):
                for j in range(i + 1, n):
                    d = delta_2opt(ruta, i, j, D)
                    evals += 1
                    if d < mejor_d - 1e-12:
                        mejor_d, mi, mj = d, i, j
            if mi is None:          # óptimo local
                break
            aplicar_2opt(ruta, mi, mj)
            c += mejor_d
            hist.append(c)
        if c < mejor_costo:
            mejor_costo, mejor_global = c, ruta[:]
        hist_global.extend(hist)
    return mejor_global, mejor_costo, hist_global, evals


def simulated_annealing_tsp(D, rng, T0=100.0, alpha=0.995, T_min=1e-3,
                            iter_por_T=50):
    """Recocido simulado con enfriamiento geométrico T <- alpha*T."""
    n = len(D)
    ruta = list(range(n))
    rng.shuffle(ruta)
    c = costo(ruta, D)
    mejor, mejor_c = ruta[:], c
    hist = [c]
    T = T0
    evals = 0
    while T > T_min:
        for _ in range(iter_por_T):
            i, j, d = vecino_aleatorio(ruta, D, rng)
            evals += 1
            if d < 0 or rng.random() < math.exp(-d / T):
                aplicar_2opt(ruta, i, j)
                c += d
                if c < mejor_c - 1e-12:
                    mejor, mejor_c = ruta[:], c
            hist.append(mejor_c)
        T *= alpha
    return mejor, mejor_c, hist, evals


# ---------------------------------------------------------------- Rastrigin
def rastrigin(x):
    return 10 * len(x) + sum(xi ** 2 - 10 * math.cos(2 * math.pi * xi) for xi in x)


def hc_rastrigin(rng, eta=0.1, iters=5000, dim=2):
    x = [rng.uniform(-5.12, 5.12) for _ in range(dim)]
    f = rastrigin(x)
    hist = [f]
    for _ in range(iters):
        y = [min(5.12, max(-5.12, xi + rng.gauss(0, eta))) for xi in x]
        fy = rastrigin(y)
        if fy < f:
            x, f = y, fy
        hist.append(f)
    return x, f, hist


def sa_rastrigin(rng, eta=0.5, T0=10.0, alpha=0.999, iters=5000, dim=2):
    x = [rng.uniform(-5.12, 5.12) for _ in range(dim)]
    f = rastrigin(x)
    mejor, mf = x[:], f
    hist = [mf]
    T = T0
    for _ in range(iters):
        y = [min(5.12, max(-5.12, xi + rng.gauss(0, eta))) for xi in x]
        fy = rastrigin(y)
        if fy < f or rng.random() < math.exp(-(fy - f) / max(T, 1e-12)):
            x, f = y, fy
            if f < mf:
                mejor, mf = x[:], f
        hist.append(mf)
        T *= alpha
    return mejor, mf, hist


# ---------------------------------------------------------------- Experimentos
def graficar_ruta(ax, ciudades, ruta, titulo, color):
    p = ciudades[ruta + [ruta[0]]]
    ax.plot(p[:, 0], p[:, 1], "-", color=color, lw=1.5, zorder=1)
    ax.scatter(ciudades[:, 0], ciudades[:, 1], c="white", edgecolors=GRIS, s=45, zorder=2)
```

```
    ax.scatter(*ciudades[ruta[0]], c="black", s=70, zorder=3)
    ax.set_title(titulo, fontsize=10)
    ax.set_aspect("equal")
    ax.tick_params(labelsize=7)


def main():
    ciudades = generar_ciudades(N_CIUDADES)
    D = matriz_distancias(ciudades)
    res = {"semilla": SEMILLA, "n_ciudades": N_CIUDADES, "corridas": N_CORRIDAS}

    # ---- 30 corridas por algoritmo
    datos = {"HC": [], "HC_reinicios": [], "SA": []}
    mejores = {}
    hist_rep = {}
    tiempos = {"HC": 0, "HC_reinicios": 0, "SA": 0}
    evals_tot = {"HC": 0, "HC_reinicios": 0, "SA": 0}
    for k in range(N_CORRIDAS):
        for nombre, fn in (
            ("HC", lambda r: hill_climbing_tsp(D, r)),
            ("HC_reinicios", lambda r: hill_climbing_tsp(D, r, reinicios=10)),
            ("SA", lambda r: simulated_annealing_tsp(D, r)),
        ):
            rng = random.Random(SEMILLA * 1000 + k)
            t0 = time.perf_counter()
            ruta, c, hist, ev = fn(rng)
            tiempos[nombre] += time.perf_counter() - t0
            evals_tot[nombre] += ev
            datos[nombre].append(c)
            if nombre not in mejores or c < mejores[nombre][1]:
                mejores[nombre] = (ruta, c)
                hist_rep[nombre] = hist

    resumen = {}
    for n, v in datos.items():
        v = np.array(v)
        resumen[n] = {
            "mejor": float(v.min()), "peor": float(v.max()),
            "media": float(v.mean()), "desv": float(v.std(ddof=1)),
            "mediana": float(np.median(v)),
            "tiempo_medio_s": tiempos[n] / N_CORRIDAS,
            "evals_medias": evals_tot[n] / N_CORRIDAS,
        }
    res["tsp"] = resumen
    res["tsp_mejores_rutas"] = {k: [int(x) for x in v[0]] for k, v in mejores.items()}

    # ---- Gráfica: rutas
    fig, axs = plt.subplots(1, 3, figsize=(12, 4.2))
    etiquetas = {"HC": "Hill Climbing", "HC_reinicios": "HC + 10 reinicios",
                 "SA": "Simulated Annealing"}
    colores = {"HC": GRIS, "HC_reinicios": VERDE, "SA": AZUL}
    for ax, n in zip(axs, ["HC", "HC_reinicios", "SA"]):
        graficar_ruta(ax, ciudades, mejores[n][0],
                      f"{etiquetas[n]}\nmejor costo = {mejores[n][1]:.1f}", colores[n])
    plt.tight_layout()
    plt.savefig(f"{OUT}/rutas.png", dpi=160)
    plt.close()

    # ---- Gráfica: convergencia (mejor corrida)
    fig, ax = plt.subplots(1, 2, figsize=(12, 4))
    ax[0].plot(hist_rep["HC"], color=GRIS, label="Hill Climbing")
    ax[0].set_xlabel("Iteración (movimiento aceptado)")
    ax[0].set_ylabel("Costo de la ruta")
    ax[0].set_title("Convergencia - Hill Climbing")
    ax[0].grid(alpha=.3)
    ax[0].legend()
    ax[1].plot(hist_rep["SA"], color=AZUL, label="Simulated Annealing (mejor hallado)")
    ax[1].set_xlabel("Evaluación de vecino")
    ax[1].set_ylabel("Mejor costo")
    ax[1].set_title("Convergencia - Simulated Annealing")
    ax[1].grid(alpha=.3)
    ax[1].legend()
    plt.tight_layout()
    plt.savefig(f"{OUT}/convergencia.png", dpi=160)
    plt.close()

    # ---- Gráfica: boxplot comparación
    fig, ax = plt.subplots(figsize=(6.5, 4))
    bp = ax.boxplot([datos["HC"], datos["HC_reinicios"], datos["SA"]],
                    patch_artist=True, medianprops=dict(color="black", lw=2), tick_labels=["HC", "HC+reinicios", "SA"])
    for patch, col in zip(bp["boxes"], [GRIS, VERDE, AZUL]):
        patch.set_facecolor(col)
        patch.set_alpha(.6)
    ax.set_ylabel("Costo final (30 corridas)")
    ax.set_title("Comparación de resultados - TSP 25 ciudades")
```

```
    ax.grid(axis="y", alpha=.3)
    plt.tight_layout()
    plt.savefig(f"{OUT}/comparacion.png", dpi=160)
    plt.close()

    # ---- Sensibilidad de parámetros SA (T0, alpha), 10 corridas c/u
    T0s = [1, 10, 100]
    alphas = [0.90, 0.95, 0.99, 0.995]
    sens = {}
    for T0 in T0s:
        for a in alphas:
            vals = []
            for k in range(10):
                rng = random.Random(SEMILLA * 7 + k)
                _, c, _, _ = simulated_annealing_tsp(D, rng, T0=T0, alpha=a,
                                                     iter_por_T=30)
                vals.append(c)
            sens[f"T0={T0}, alpha={a}"] = {"media": float(np.mean(vals)),
                                           "mejor": float(np.min(vals))}
    res["sensibilidad_SA"] = sens
    M = np.array([[sens[f"T0={T0}, alpha={a}"]["media"] for a in alphas] for T0 in T0s])
    fig, ax = plt.subplots(figsize=(6, 3.6))
    im = ax.imshow(M, cmap="Greys_r", aspect="auto")
    ax.set_xticks(range(len(alphas)), [str(a) for a in alphas])
    ax.set_yticks(range(len(T0s)), [str(t) for t in T0s])
    ax.set_xlabel("alpha (enfriamiento)")
    ax.set_ylabel("Temperatura inicial T0")
    ax.set_title("Costo medio de SA según parámetros")
    for i in range(M.shape[0]):
        for j in range(M.shape[1]):
            ax.text(j, i, f"{M[i, j]:.0f}", ha="center", va="center",
                    color="white" if M[i, j] < M.mean() else "black", fontsize=9)
    plt.colorbar(im, ax=ax, label="costo medio")
    plt.tight_layout()
    plt.savefig(f"{OUT}/sensibilidad_sa.png", dpi=160)
    plt.close()

    # ---- Rastrigin (η)
    etas = [0.05, 0.1, 0.5, 1.0]
    ras = {}
    for eta in etas:
        hc_v = [hc_rastrigin(random.Random(SEMILLA + k), eta=eta)[1] for k in range(N_CORRIDAS)]
        sa_v = [sa_rastrigin(random.Random(SEMILLA + k), eta=eta)[1] for k in range(N_CORRIDAS)]
        ras[str(eta)] = {"HC_media": float(np.mean(hc_v)), "HC_mejor": float(np.min(hc_v)),
                         "SA_media": float(np.mean(sa_v)), "SA_mejor": float(np.min(sa_v))}
    res["rastrigin"] = ras
    _, _, h_hc = hc_rastrigin(random.Random(SEMILLA), eta=0.1)
    _, _, h_sa = sa_rastrigin(random.Random(SEMILLA), eta=0.5)
    fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
    ax[0].plot(h_hc, color=GRIS, label="HC (η=0.1)")
    ax[0].plot(h_sa, color=AZUL, label="SA (η=0.5)")
    ax[0].set_xlabel("Iteración")
    ax[0].set_ylabel("Mejor f(x)")
    ax[0].set_title("Rastrigin 2D - convergencia")
    ax[0].legend()
    ax[0].grid(alpha=.3)
    w = 0.35
    xs = np.arange(len(etas))
    ax[1].bar(xs - w / 2, [ras[str(e)]["HC_media"] for e in etas], w, color=GRIS, label="HC")
    ax[1].bar(xs + w / 2, [ras[str(e)]["SA_media"] for e in etas], w, color=AZUL, label="SA")
    ax[1].set_xticks(xs, [str(e) for e in etas])
    ax[1].set_xlabel("η (desviación del paso)")
    ax[1].set_ylabel("f media (30 corridas)")
    ax[1].set_title("Efecto de η")
    ax[1].legend()
    ax[1].grid(axis="y", alpha=.3)
    plt.tight_layout()
    plt.savefig(f"{OUT}/rastrigin.png", dpi=160)
    plt.close()

    with open(f"{OUT}/resultados.json", "w", encoding="utf-8") as f:
        json.dump(res, f, indent=2, ensure_ascii=False)

    print(json.dumps({"tsp": resumen, "rastrigin": ras}, indent=2))


if __name__ == "__main__":
    main()
```

4.1 Código de la simulación animada

Archivo `simulacion.py`. Importa las funciones de `busqueda_local.py`, ejecuta HC y SA con la semilla 171 guardando instantáneas de la ruta y genera `resultados/simulacion_tsp.gif` (animación) y `resultados/simulacion_cuadros.png`.

```
"""
Simulación animada de Hill Climbing y Simulated Annealing sobre el TSP (25 ciudades).
Reutiliza las funciones de busqueda_local.py y genera, en resultados/:
  - simulacion_tsp.gif        animación (ruta actual y curva de costo de ambos algoritmos)
  - simulacion_cuadros.png    cuadros fijos (inicio, 25 %, 60 % y final) para el informe

Uso:  python simulacion.py
"""
import math
import os
import random

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.animation import FuncAnimation, PillowWriter

from busqueda_local import (SEMILLA, N_CIUDADES, OUT, aplicar_2opt, costo, delta_2opt,
                            generar_ciudades, matriz_distancias, vecino_aleatorio)

N_FRAMES = 60
CHECK_SA = 1000          # cada cuántas evaluaciones se guarda una instantánea de SA


def trazar_hc(D, rng):
    """Hill Climbing steepest-ascent: guarda la ruta tras cada movimiento aceptado."""
    n = len(D)
    ruta = list(range(n))
    rng.shuffle(ruta)
    c = costo(ruta, D)
    snaps = [(ruta[:], c, None)]
    while True:
        mejor_d, mi, mj = 0.0, None, None
        for i in range(1, n - 1):
            for j in range(i + 1, n):
                d = delta_2opt(ruta, i, j, D)
                if d < mejor_d - 1e-12:
                    mejor_d, mi, mj = d, i, j
        if mi is None:
            break
        aplicar_2opt(ruta, mi, mj)
        c += mejor_d
        snaps.append((ruta[:], c, None))
    return snaps


def trazar_sa(D, rng, T0=100.0, alpha=0.995, T_min=1e-3, iter_por_T=50):
    """Simulated Annealing: guarda ruta actual, costo actual y T cada CHECK_SA evaluaciones."""
    n = len(D)
    ruta = list(range(n))
    rng.shuffle(ruta)
    c = costo(ruta, D)
    snaps = [(ruta[:], c, T0)]
    T, evals = T0, 0
    while T > T_min:
        for _ in range(iter_por_T):
            i, j, d = vecino_aleatorio(ruta, D, rng)
            evals += 1
            if d < 0 or rng.random() < math.exp(-d / T):
                aplicar_2opt(ruta, i, j)
                c += d
            if evals % CHECK_SA == 0:
                snaps.append((ruta[:], c, T))
        T *= alpha
    snaps.append((ruta[:], c, T))
    return snaps


def indice(k, total, n_frames=N_FRAMES):
    return int(round(k / (n_frames - 1) * (total - 1)))


def dibujar_ruta(ax, ciudades, ruta, titulo):
    ax.clear()
    p = ciudades[ruta + [ruta[0]]]
```

```
    ax.plot(p[:, 0], p[:, 1], "-", color="black", lw=1.3, zorder=1)
    ax.scatter(ciudades[:, 0], ciudades[:, 1], c="white", edgecolors="black", s=40, zorder=2)
    ax.scatter(*ciudades[ruta[0]], c="black", s=65, zorder=3)
    ax.set_title(titulo, fontsize=9)
    ax.set_aspect("equal")
    ax.set_xlim(-5, 105)
    ax.set_ylim(-5, 105)
    ax.tick_params(labelsize=6)


def main():
    ciudades = generar_ciudades(N_CIUDADES)
    D = matriz_distancias(ciudades)
    hc = trazar_hc(D, random.Random(SEMILLA))
    sa = trazar_sa(D, random.Random(SEMILLA))
    c_hc = [s[1] for s in hc]
    c_sa = [s[1] for s in sa]
    print(f"HC: {len(hc)} instantáneas, costo final {c_hc[-1]:.2f}")
    print(f"SA: {len(sa)} instantáneas, costo final {c_sa[-1]:.2f}")

    def titulo_hc(k):
        i = indice(k, len(hc))
        return f"Hill Climbing · movimiento {i} · costo {hc[i][1]:.1f}"

    def titulo_sa(k):
        i = indice(k, len(sa))
        return f"Simulated Annealing · T = {sa[i][2]:.2f} · costo {sa[i][1]:.1f}"

    # ------------------------------------------------ GIF
    fig, axs = plt.subplots(2, 2, figsize=(9, 7.2), gridspec_kw={"height_ratios": [3, 1.6]})
    ax_hc, ax_sa, ax_c1, ax_c2 = axs[0, 0], axs[0, 1], axs[1, 0], axs[1, 1]
    lim_y = (min(c_sa + c_hc) - 30, max(c_sa + c_hc) + 30)

    def actualizar(k):
        ih, is_ = indice(k, len(hc)), indice(k, len(sa))
        dibujar_ruta(ax_hc, ciudades, hc[ih][0], titulo_hc(k))
        dibujar_ruta(ax_sa, ciudades, sa[is_][0], titulo_sa(k))
        for ax, costos, i, nom, xl in ((ax_c1, c_hc, ih, "Hill Climbing", "Movimiento aceptado"),
                                       (ax_c2, c_sa, is_, "Simulated Annealing", f"Instantánea (cada {CHECK_SA} evaluaciones)")):
            ax.clear()
            ax.plot(range(len(costos)), costos, color="0.75", lw=1)
            ax.plot(range(i + 1), costos[:i + 1], color="black", lw=1.6)
            ax.set_xlim(0, len(costos) - 1)
            ax.set_ylim(*lim_y)
            ax.set_xlabel(xl, fontsize=7)
            ax.set_ylabel("Costo", fontsize=7)
            ax.grid(alpha=.3)
            ax.tick_params(labelsize=6)
        fig.tight_layout()

    anim = FuncAnimation(fig, actualizar, frames=N_FRAMES)
    anim.save(f"{OUT}/simulacion_tsp.gif", writer=PillowWriter(fps=8))
    plt.close(fig)

    # ------------------------------------------------ cuadros fijos para el PDF
    pasos = [0, 0.25, 0.6, 1.0]
    fig, axs = plt.subplots(2, 4, figsize=(12, 6.2))
    for col, f in enumerate(pasos):
        k = int(round(f * (N_FRAMES - 1)))
        ih, is_ = indice(k, len(hc)), indice(k, len(sa))
        dibujar_ruta(axs[0, col], ciudades, hc[ih][0], f"HC · mov. {ih}\ncosto {hc[ih][1]:.1f}")
        dibujar_ruta(axs[1, col], ciudades, sa[is_][0], f"SA · T = {sa[is_][2]:.2f}\ncosto {sa[is_][1]:.1f}")
    plt.tight_layout()
    plt.savefig(f"{OUT}/simulacion_cuadros.png", dpi=160)
    plt.close()


if __name__ == "__main__":
    main()
```

5. Resultados

5.1 TSP: comparación sobre 30 corridas independientes

Algoritmo	Mejor	Media	Mediana	Peor	Desv. est.	Evaluaciones (media)	Tiempo medio
Hill Climbing	420.37	434.41	432.51	465.20	11.35	5 630	4 ms
HC + 10 reinicios	420.37	421.30	420.37	426.30	1.82	56 010	41 ms
Simulated Annealing	420.37	421.96	420.37	428.12	2.57	114 850	362 ms

Tabla 1. Costo (longitud de la ruta) tras 30 corridas con semillas distintas.

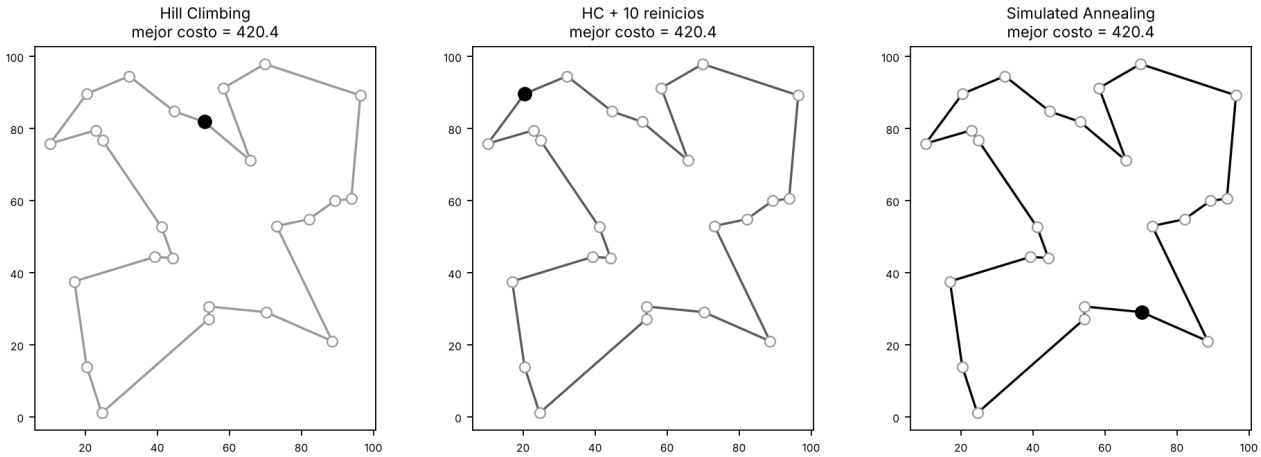


Figura 1. Mejor ruta hallada por cada algoritmo (el punto negro grande es la ciudad de inicio).

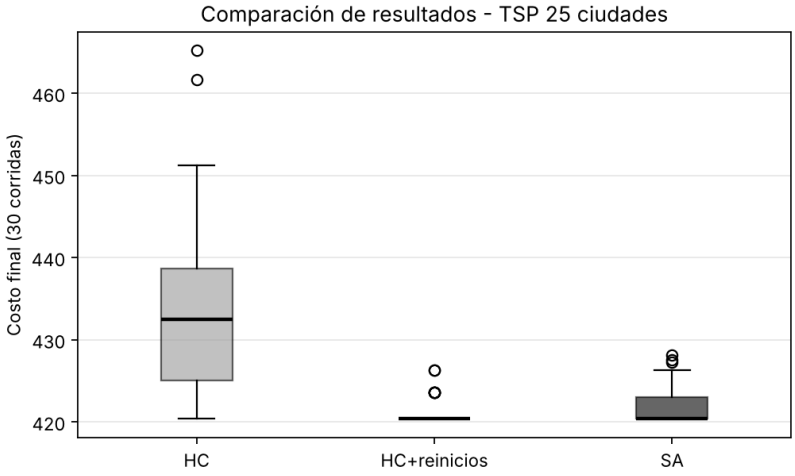


Figura 2. Distribución del costo final en 30 corridas.

5.2 Convergencia

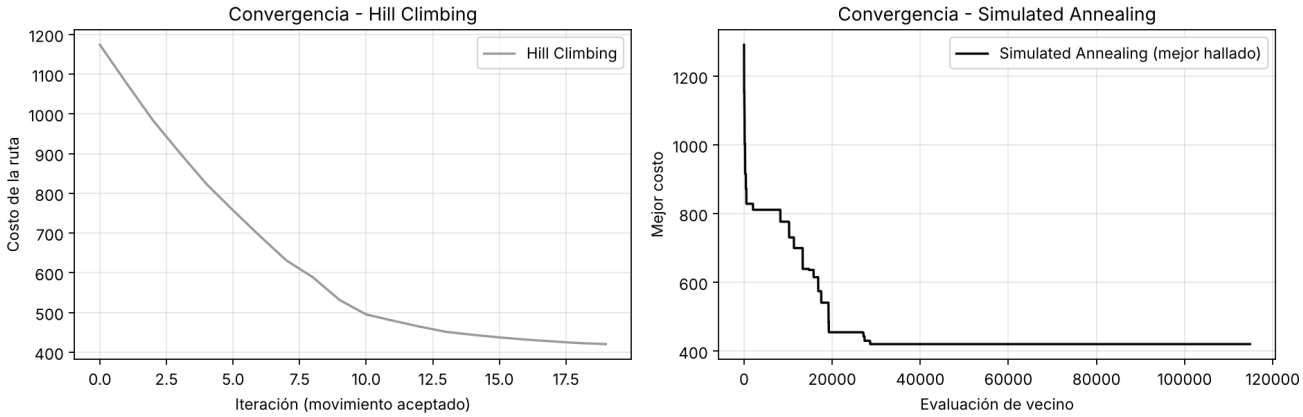


Figura 3. Convergencia de la mejor corrida. HC desciende de ≈1 180 a un óptimo local en unos 20 movimientos; SA mejora por escalones mientras T es alta y se estabiliza cerca de la evaluación 28 000.

5.3 Sensibilidad de SA a T_0 y α (10 corridas por combinación, 30 iteraciones por temperatura)

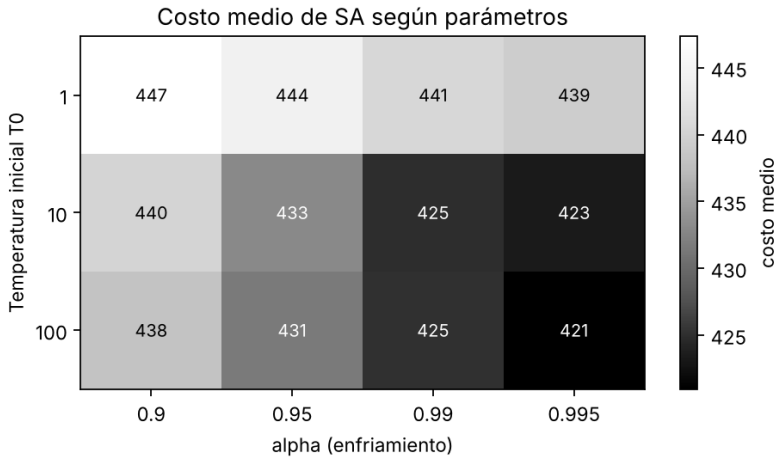


Figura 4. Costo medio de SA. Valores menores (más oscuros) son mejores.

T_0 \ α	0.90	0.95	0.99	0.995
1	447.4	444.5	440.5	439.4
10	440.2	433.0	424.9	423.4
100	438.3	431.5	425.2	421.0

Tabla 2. Costo medio de SA (mejor valor posible hallado: 420.37).

5.4 Rastrigin 2D: efecto del tamaño de paso η (30 corridas)

η	HC media	HC mejor	SA media	SA mejor
0.05	18.042	0.9950	12.189	0.9950
0.1	17.379	0.9950	8.048	0.9950
0.5	0.019	0.0017	0.047	0.0002
1.0	0.080	0.0009	0.145	0.0071

Tabla 3. Valor final de f(x); el óptimo global es 0.

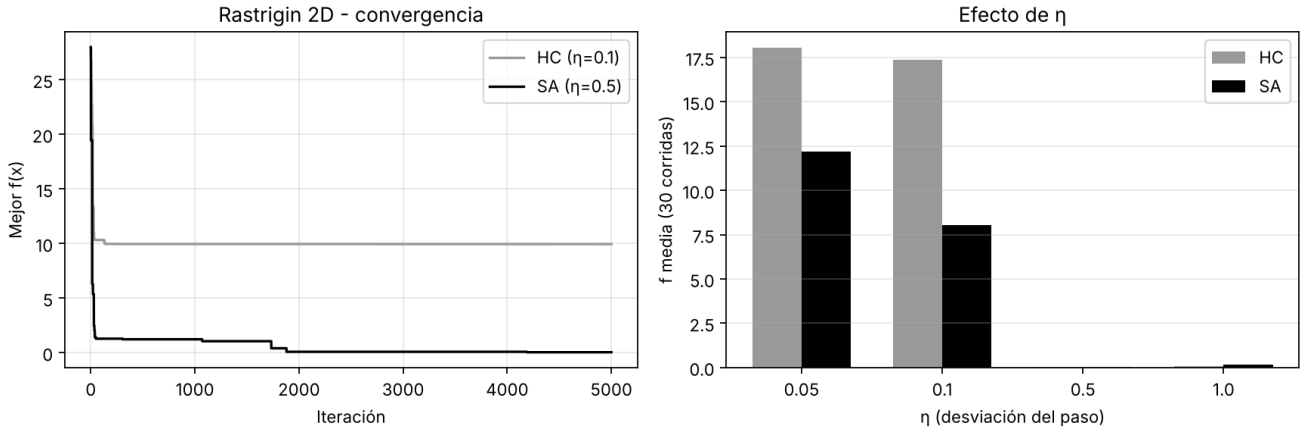


Figura 5. Convergencia (izquierda) y efecto de η sobre la media (derecha).

5.5 Simulación animada

La animación `simulacion_tsp.gif` (incluida en el repositorio) muestra, lado a lado, cómo cada algoritmo modifica la ruta y cómo evoluciona su costo. La Figura 6 reúne cuatro cuadros de esa animación (inicio, 25 %, 60 % y final) para una ejecución con la semilla 171.

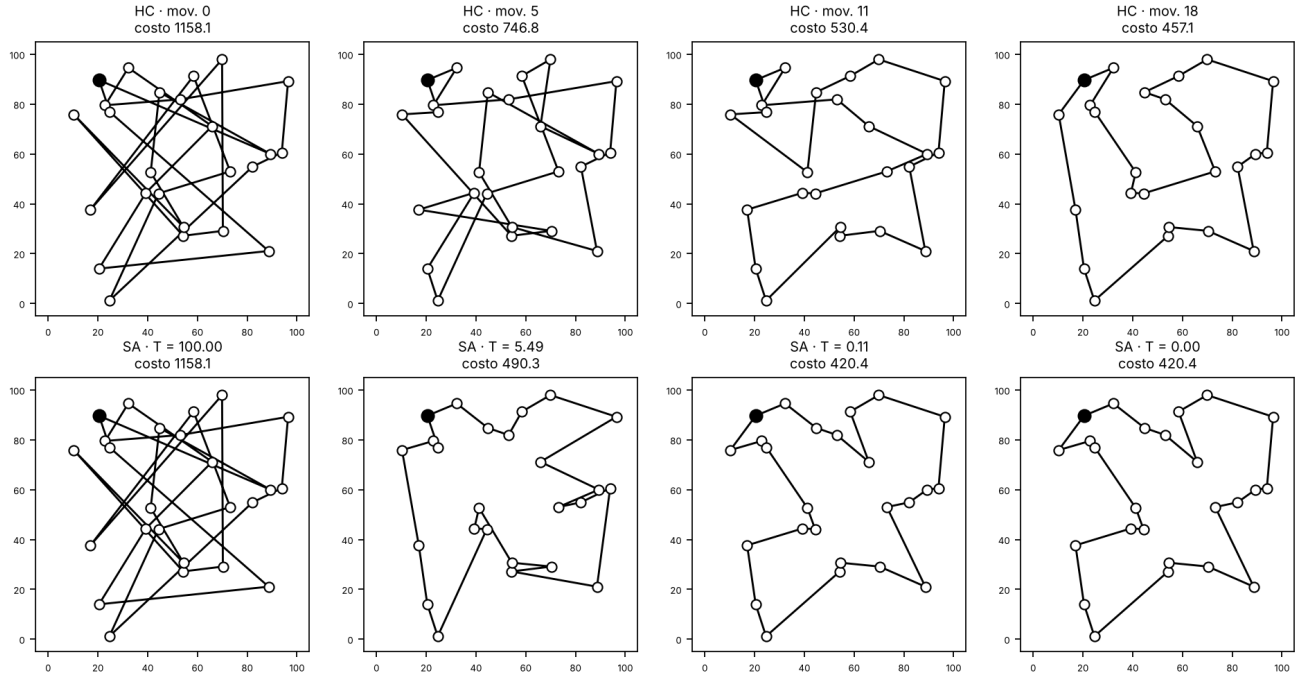


Figura 6. Cuadros de la simulación. Arriba: Hill Climbing (movimientos aceptados). Abajo: Simulated Annealing (temperatura T).

En esta ejecución, HC mejoró la ruta de forma rápida y constante, pero se detuvo en un óptimo local con costo 457.10 tras 18 movimientos. SA aceptó movimientos peores mientras T era alta, por lo que la ruta se ve desordenada al inicio, y al enfriarse se estabilizó en 420.37. Se trata de una sola ejecución ilustrativa; la comparación estadística está en la sección 5.1.

6. Discusión y conclusiones

- **HC simple se atrapa en óptimos locales.** Con una sola ejecución, su media fue 434.4 con desviación de 11.4, y su peor corrida (465.2) quedó ≈ 10.7 % por encima del mejor valor conocido (420.37). Es el más rápido (~4 ms) pero el menos fiable.
- **SA y HC con reinicios alcanzan calidad similar.** Ambos encontraron 420.37 como mejor valor y su mediana coincide con él (media 421.30 y 421.96). Con 25 ciudades, HC con 10 reinicios lo logra con la mitad de evaluaciones (≈56 000 contra ≈115 000) y es unas 9 veces más rápido en tiempo real, así que no se puede afirmar que SA sea superior en este tamaño de problema.
- **Los parámetros de SA importan.** Con $T_0 = 1$ el algoritmo se parece a HC y no mejora y mejora poco aunque se enfríe más lento; con $T_0 \geq 10$ y $\alpha \geq 0.99$ la media baja a 421–425. Un enfriamiento rápido ($\alpha = 0.90$) deja resultados más pobres.
- **η en Rastrigin.** Con $\eta = 0.05$ y 0.1 ambos métodos quedan atrapados en mínimos locales ($f \approx 1$); SA los supera por el criterio de aceptación (8.0 contra 17.4 con $\eta = 0.1$). Con $\eta = 0.5$, el paso es suficientemente grande para saltar entre valles y HC logró incluso mejor media que SA (0.019 contra 0.047), lo que muestra que el tamaño de paso puede pesar más que la regla de aceptación.

- **Limitaciones.** Se usó una sola instancia de 25 ciudades y no se verificó el óptimo global con un método exacto; los resultados podrían cambiar con instancias más grandes, donde SA suele ganar ventaja.

7. Declaración de uso de IA

Herramienta	Claude Sonnet 5.5, modelo de lenguaje de Anthropic (Anthropic, 2026)
Prompt	Se le entregó la hoja de la actividad y se pidió realizar la actividad de la Sección 3 (Búsqueda Local), en blanco y negro y con la portada y el tema que pide la hoja.
Uso	Generación del código Python, de los experimentos, las gráficas y el borrador de este informe (Anthropic, 2026).
Verificación	El código se ejecutó y los números del informe provienen de esa ejecución (resultados.json). La alumna debe revisar el código, comprender cada parte y ajustar el texto antes de entregar.

8. Referencias (APA 7.ª ed.)

Anthropic. (2026). *Claude Sonnet 5.5* [Modelo de lenguaje de gran tamaño]. https://claude.ai

Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research, 6*(6), 791–812. https://doi.org/10.1287/opre.6.6.791

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science, 220*(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

Russell, S., & Norvig, P. (2020). *Inteligencia Artificial: Un enfoque moderno* (4.ª ed.). Pearson. https://www.pearson.com/en-us/subject-catalog/p/artificial-intelligence-a-modern-approach/P200000003500

Wirsansky, E. (2020). *Hands-On Genetic Algorithms with Python*. Packt. https://www.packtpub.com/product/hands-on-genetic-algorithms-with-python/9781838557744